SURF · SNELLIUS

Using Snellius

SBUs, nodes, storage and your first job

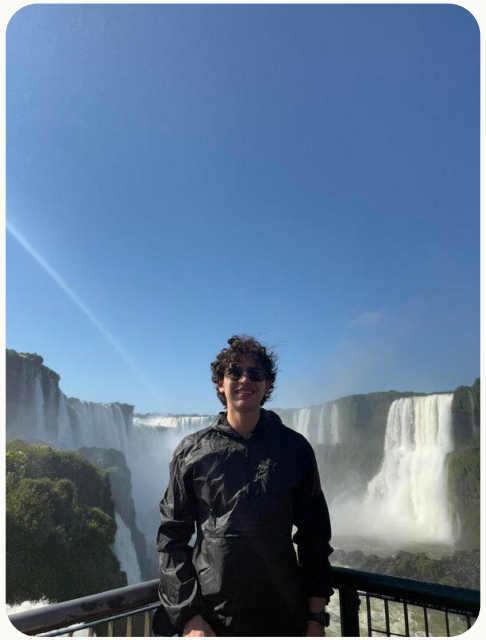
Based on the SURF partitions page and the UvA DL cluster tutorial

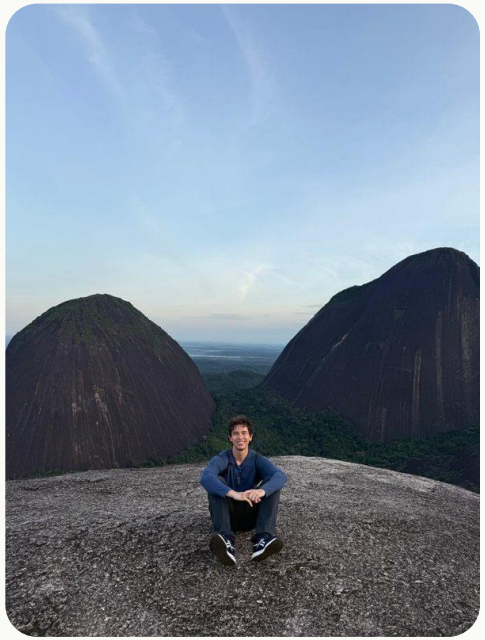
NICE TO MEET YOU

Hi! I'm Daniel

From Medellín, Colombia

Your TA for Deep Learning 1






The seven-week plan

W1	**Snellius 101**	Today
W2	**Research workflow in AI**	Start local, then scale up
W3	**Implementing convolutions**	Early CNNs to ConvNeXt
W4	**Transformer inputs and outputs**	Tensor shapes, ViTs in code
W5	**Self-supervised learning**	The state of the art
W6	**Advanced generative learning**	Flow matching, Drifting
W7	**Exam prep**	Past exams, together

TODAY

1 SBUs and your budget

2 Nodes and storage

3 Everyday commands

4 Environments and modules

5 Job files

6 Good practices

PART 1 OF 6

SBUs

The Snellius currency: what a job costs.

PART 1 · THE CURRENCY

What is an SBU?

System Billing Unit. Every job spends them from your budget.

```
SBUs = resources × hours × weight
```

1	128	192
CPU core-hour on a standard node	A100 GPU-hours, each	H100 GPU-hours, each

> A whole A100 node is 512 an hour: 4 GPUs × 128.

BEST WAY TO MANAGE YOUR SBUS

Never run an agent in the cluster.

You are responsible.

If an agent burns the whole course's credits, you pay for it. Not your agent.

PART 2 OF 6

Nodes and storage

Where jobs run, where you land, where files live.

CPU nodes

What you will use them for, then the specs.

What you will use them for

`Download data`	Fetch datasets into scratch.
`Process data`	Clean, convert, tokenize.
`CPU-only RL`	Environments that run on CPU.
`Classic ML`	Most of scikit-learn.

Use rome or genoa. Out of memory? fat or himem.

CPU nodes

`rome`	Default. Preprocessing, simulations, many small jobs.
`genoa`	Newer, 192 cores. Big parallel jobs on whole nodes.
`fat_*`	Out of memory on rome? E.g. a 300 GB table in RAM.
`himem_4tb`	Fat is too small. E.g. genome assembly, huge indexes.
`himem_8tb`	Rare. Only when 4 TB is not enough.
`staging`	Data transfers, e.g. from the Archive. Not for compute.

Tests under 1 hour start sooner. Next: what the smallest request costs.

Smallest ask and weight

PARTITION	SMALLEST ASK	WHOLE NODE	WEIGHT
`rome`	16 cores + 28 GiB	128 cores	**1.0**
`genoa`	24 cores + 42 GiB	192 cores	**1.0**
`fat_rome`	16 cores + 120 GiB	128 cores	**1.5**
`fat_genoa`	24 cores + 180 GiB	192 cores	**1.5**
`himem_4tb`	16 cores + 480 GiB	128 cores	**2.0**
`himem_8tb`	16 cores + 960 GiB	128 cores	**3.0**

Smallest ask: 1/8 of a node Maximum run time: 120 hours

PART 2 · BILLING

You pay for what you ask for

A CPU node is split in 8. One rome slice is **16 cores, 28 GiB**.

`1 core` Still reserves one slice		**16** SBU / hour
`16 cores` Fills one slice		**16** SBU / hour
`17 cores` One core over a slice		**32** SBU / hour
`1 core + 30 GiB` Memory counts too		**32** SBU / hour

You pay for the bigger of cores and memory, in whole slices.

GPU nodes

What you will use them for, then the specs.

GPU nodes

`gpu_mig`	A slice of an A100. Debugging, small experiments, course work.
`gpu_a100`	40 GiB. The workhorse for real training and fine-tuning.
`gpu_h100`	94 GiB, faster. Models that do not fit in 40 GiB.
`gpu_vis`	Visualisation and rendering. Not for training.

Start small, move up only when the job needs it.

Smallest ask and weight

PARTITION	SMALLEST ASK	WHOLE NODE	SBU/H
gpu_mig	9 cores + 1 MIG + 60 GiB	72 cores, 8 GPUs (MIG)	64
gpu_a100	18 cores + 1 GPU + 120 GiB	72 cores, 4 GPUs	128
gpu_h100	16 cores + 1 GPU + 180 GiB	64 cores, 4 GPUs	192
gpu_vis	18 cores + 1 GPU + 120 GiB	72 cores, 4 GPUs	128

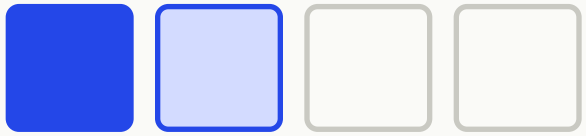
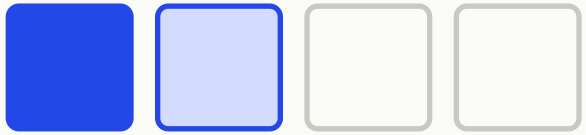
Smallest ask: 1/4 of a node Maximum run time: 120 hours

MIG is 1/8 of a node and gpu_vis is 24 hours. Cores come with the GPU: ask for more and you pay for a bigger share.

PART 2 · BILLING

You pay for what you ask for

A GPU node is split in 4. One A100 slice is **1 GPU, 18 cores, 120 GiB**.

`1 GPU + 18 cores` Fits in one slice		**128** SBU / hour
`1 GPU + 32 cores` More cores than a slice		**256** SBU / hour
`1 GPU + 130 GiB` More memory than a slice		**256** SBU / hour

Over one slice, cores or memory: twice the price, same single GPU.

PART 2 · YOUR BUDGET

Checking SBUs

`accinfo`

Your budget and what is left

`accuse`

Who spent what

`budget-overview -p gpu`

GPU budget at a glance

```
sacct -j JOBID -o "JobID,JobName,MaxRSS,Elapsed,NNodes,NodeList"
```

What one finished job used

PART 2 · TESTING

Testing? Ask for 1 hour or less.

```
#SBATCH --time=00:30:00
```

WHY	Short jobs run on nodes kept for them, so they start sooner.
WHERE	Thin, fat and GPU partitions.
CAREFUL	Only a few such nodes. Ask for one, not tens.

Login nodes

Where you land, and what to do there.

Login nodes

3 **CPU-only machines, shared by everyone at once.**

Copying data: Transfer files between systems or directories

PART 2 · WHERE YOU LAND

Login nodes

3 **CPU-only machines, shared by everyone at once.**

DON'T

Train

Process or run anything heavy

Run anything over 15 min: it gets killed

Copying data: Transfer files between systems or directories

Login nodes

3 **CPU-only machines, shared by everyone at once.**

DO

Edit files

Move data

Submit and watch jobs

Copying data: Transfer files between systems or directories

Storage

Where files live, and for how long.

PART 2 · WHERE YOUR FILES LIVE

Five places to keep files

`home`

200 GiB

Never deleted

`scratch-shared`

8 TiB

14 days

`scratch-local`

Same 8 TiB

6 days

`scratch-node`

Your slice

Until the job ends

`project space`

On request

Project life

PART 2 · WHERE YOUR FILES LIVE

Local is not always local

CENTRAL NETWORK STORAGE

`scratch-shared`

Every node sees it.

`scratch-local`

Private to one node, by software.

INSIDE THE NODE

`scratch-node`

A real NVMe disk in the machine.

Only scratch-node is a disk inside the machine. The other two share the network with everyone.

One variable, two places

$TMPDIR
A path your job writes to

no flag →

scratch-local
A folder on network storage.

$TMPDIR
A path your job writes to

`--constraint=scratch-node` →

scratch-node
The disk inside the node.

Your code always writes to $TMPDIR. The flag decides where that path lands.

How much space do you need?

How fast must the GPU read it?

How much space?

`home`	Code and results you keep.	**200 GiB**
`scratch-shared`	Datasets every node can read.	**8 TiB**
`scratch-local`	Counts against the same 8 TiB.	**Same 8 TiB**
`scratch-node`	Only your slice of the node disk.	**Your slice**

The node disk is only your slice. Bigger data goes to scratch.

How fast?

`scratch-shared`	Every user on every node competes.	**Crowded**
`scratch-local`	Per node, so fewer users competing.	**In between**
`scratch-node`	Your own disk. No competition.	**Fastest**

Too big for your NVMe slice, and no wish to pay for a bigger node? Use scratch-local.

PART 2 · WHERE YOUR FILES LIVE

The three scratch spaces

	scratch-shared	scratch-local	scratch-node
WHERE	Network	Network	Node disk
SEEN BY	All nodes	One node	One job
QUOTA	8 TiB	8 TiB	None
KEPT	14 days	6 days	Job end
SPEED	Varies	Varies	Fastest

Using the spaces

How to move around, and how scratch-local and scratch-node work.

PART 2 · WHERE YOUR FILES LIVE

Moving around

MAKE YOURS	`mkdir -p /scratch-shared/$USER`
GO THERE	`cd /scratch-shared/$USER`
COPY IN	`cp -r ~/data /scratch-shared/$USER/`
GO HOME	`cd ~`

$USER is your username, so every student gets a private folder.

$TMPDIR in a job script

CHOOSE	`#SBATCH --constraint=scratch-node`	Needed for the speed-up.
COPY IN	`cp -r /scratch-shared/$USER/data $TMPDIR/`	Data lands on the job's disk.
RUN	`python train.py --data $TMPDIR/data`	Read and write there.
COPY OUT	`cp -r $TMPDIR/results $HOME/`	Before the job ends.

Without the flag, $TMPDIR is still network storage. Check space: df -h $TMPDIR.

PART 2 · WHERE YOUR FILES LIVE

How much node disk you get

1 GPU a quarter of the node		**1.6 TB**
Half a node e.g. 32 cores		**3.2 TB**
Whole node 4 GPUs		**6.4 TB**

Each node has a 6.4 TB NVMe disk. You get the same share of it as of the node.

PART 2 · WHERE YOUR FILES LIVE

Four habits

STORE	`wget -P /scratch-shared/$USER <url>`	Big data goes here, not in home.
CHECK	`myquota`	Know your 8 TiB before a big job.
SPEED	`#SBATCH --constraint=scratch-node`	When the GPU waits on data reads.
SAVE	`cp -r $TMPDIR/out $HOME`	scratch-node is wiped when the job ends.

Everyday commands

Launch a job, watch it, stop it.

Everyday commands

SUBMIT	`sbatch job.sh`
QUEUE	`squeue -u $USER`
CANCEL	`scancel <jobid>`
INSPECT	`scontrol show job <jobid>`

Handy extras

FOLLOW LOG	`tail -f slurm_output_<jobid>.out`
START TIME	`squeue -u $USER --start`
CANCEL ALL	`scancel -u $USER`
RECEIPT	`sacct -S today --format=JobID,JobName,Elapsed,State`

PART 3 · YOUR QUOTA

Checking disk space

YOUR QUOTA	`myquota`
PER FOLDER	`du -sh *`
BIGGEST FIRST	`du -sh * | sort -rh | head`
INTERACTIVE	`ncdu`

Reference: Checking disk usage · Cleaning up a home directory

PART 4 OF 6

Environments and modules

Where your libraries come from.

PART 4 · ENVIRONMENTS

Which tool, where

`uv`	Manage your packages and pin their versions.	**Laptop** local development
`Anaconda3`	The module SURF maintains. We use its conda to build the environment from your yaml.	**Snellius** run in a job
`Avoid`	Installing your own conda. We skip it.	**Not maintained** by SURF

The goal: the same PyTorch and PyG versions on your laptop and on Snellius.

PART 4 · ON YOUR LAPTOP

Why we use uv

One file	pyproject.toml holds it all.
No crashes	Conflicts show up when you add a package.
Same everywhere	uv.lock pins every version.

Install it in a job, not in the login node

`No GPU`	Login nodes have none. conda may install a CPU-only PyTorch.
`15 min`	Long commands on login nodes get killed. Big installs take longer.

Reference: Connecting to the system

PART 4 · STEP 4 · INSTALL JOB, 1 OF 2

Build it once, in a job

```
#!/bin/bash
#SBATCH --partition=gpu_mig
#SBATCH --cpus-per-task=9
#SBATCH --gpus=1
#SBATCH --time=00:10:00

module purge
module load 2025
module load Anaconda3/2025.06-1
cd "$HOME/<your_repo>"
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
conda env create -f <path_to_your_conda.yaml>
```

The two terms lines are needed the first time only. The script continues on the next slide.

PART 4 · STEP 4 · INSTALL JOB, 2 OF 2

Check the CUDA build

```
# at the end of the same script
source activate <your_env>

python -c "
import torch
print(torch.__version__)
print(torch.cuda.is_available())
print(torch.version.cuda)
print(torch.backends.cudnn.version())"
```

torch.__version__
The PyTorch version

cuda.is_available()
True: the GPU is visible

version.cuda
CUDA of this build. None: CPU-only

cudnn.version()
cuDNN inside the build

A CPU-only build installs fine and fails only later, in training.

SUBMIT `sbatch install_env.sh`

READ `cat slurm-<jobid>.out`

PART 4 · CHECK THE GPU

Can PyTorch see the GPU?

UV · ON YOUR LAPTOP

```
curl -LsSf https://astral.sh/uv/install.sh | sh
uv init hello && cd hello
uv add torch
uv run python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
# Mac: no CUDA, check MPS instead
uv run python -c "import torch; print(torch.backends.mps.is_available())"
```

ANACONDA3 · ON SNELLIUS

```
module load 2025
module load Anaconda3/2025.06-1
source activate <env_name>
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

uv is for local development: True on a Mac means MPS works. Anaconda3 is for Snellius: expect CUDA True in a GPU job.

Basic module commands

FIND IT	`module spider Anaconda3`
CHOOSE A STACK	`module load 2025`
LOAD IT	`module load Anaconda3/2025.06-1`
SEE WHAT IS LOADED	`module list`
START CLEAN	`module purge`

2025 is supported, 2024 limited, 2023 deprecated. Conda comes from the SURF-maintained Anaconda3 module.

Job files

Ask for hardware, then run your code.

PART 5 · EXAMPLE JOB FILE, 1 OF 4

Flags you always use

```
#!/bin/bash

# what you are billed for
#SBATCH --partition=gpu_a100
#SBATCH --gpus=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=18
#SBATCH --mem=60G
#SBATCH --time=00:12:00

# monitoring and debugging
#SBATCH --job-name=<your_job_name>
#SBATCH --output=slurm_output_%A.out
#SBATCH --mail-type=<BEGIN|END|FAIL|ALL>
#SBATCH --mail-user=<your_email>
```

partition, gpus
What hardware you pay for

ntasks, cpus-per-task
Keep 1 and 1, unless you know why

mem
CPU RAM, not GPU memory

time
Hard limit, then the job is killed

job-name, output
Your job name and its log file

mail-type
Pick one, or a list: BEGIN,END

Reference: Writing a job script

PART 5 · EXAMPLE JOB FILE, 2 OF 4

Careful with ntasks

Cores asked = **ntasks × cpus-per-task**

`ntasks=1` `cpus-per-task=18`	**Fits one slice**	**128 SBU/h**
`ntasks=1` `cpus-per-task=32`	**Over a slice, still 1 GPU**	**256 SBU/h**
`ntasks=4` `cpus-per-task=8`	**32 cores in total**	**256 SBU/h**

One A100 slice is 18 cores. Go over it and you pay for a bigger share.

PART 5 · EXAMPLE JOB FILE, 3 OF 4

Flags you sometimes need

`--constraint=scratch-node`	Fast local NVMe disk, see storage
`--nodes, --gpus-per-node`	Multi-node jobs only
`--error`	Errors in their own file
`--dependency`	Start after another job, see Advanced tips
`--account`	Which project pays, if you have several

PART 5 · EXAMPLE JOB FILE, 4 OF 4

Software and run

```
# software
module purge
module load 2025
module load Anaconda3/2025.06-1

# environment
cd "$HOME/<your_repo>"
source activate <your_env>

# run
python -u train.py --seed 42
```

SUBMIT `sbatch job.sh` **READ** `cat slurm_output_<jobid>.out`

PART 6 OF 6

Good practices

Habits that save budget and time.

PART 6 · GOOD PRACTICE

Spend SBUs wisely

Test small first

Under 1 hour starts sooner. Then scale up.

Ask for what you use

Check the last run with sacct.

Time limit + 20%

Tight, but not killed at the finish.

Copy results out

Scratch is cleaned after 6 or 14 days.

Cancel what is stuck

scancel early. Idle GPUs still cost.

Log as it runs

python -u shows output live.

PART 6 · GOOD PRACTICE

Get data onto scratch-shared

MAKE A FOLDER	`mkdir -p /scratch-shared/$USER/data`
FROM HOME	`rsync -avh ~/data/ /scratch-shared/$USER/data/`
FROM LAPTOP	`rsync -avh --progress ./data/ <user>@snellius.surf.nl:/scratch-shared/<user>/data/`
CHECK SIZE	`du -sh /scratch-shared/$USER/data`

Reference: Transfer files between systems or directories

scp or rsync?

	scp	rsync
BEST FOR	One-off, small files	Big datasets, many files
IF IT BREAKS	Start over from 0%	Resume where it stopped
COPIES	Everything, every time	Only what changed

When in doubt, use rsync.

PART 6 · GOOD PRACTICE

Copy in, run, copy out

```
# inside your job script
#SBATCH --constraint=scratch-node # optional: NVMe

cp -r /scratch-shared/$USER/data "$TMPDIR/"
python train.py --data "$TMPDIR/data"
cp -r "$TMPDIR/results" "$HOME/runs/"
```

$TMPDIR is disk, not RAM. Check it fits: du -sh, df -h.

With the flag, $TMPDIR is the node's NVMe. Wiped at job end.

Results to home before the job ends. Scratch is not backed up.

BONUS

Advanced tips

For later, when you need many runs.

Debug live, chain jobs

Debug live: try a fix on a real GPU without queueing again.

```
srun --partition=gpu_mig --gpus=1 --time=00:30:00 --pty bash
```

Train, then evaluate: start B only if A succeeded.

```
jid=$(sbatch --parsable first.sh)
sbatch --dependency=afterok:$jid second.sh
```

Always clean up: run B however A ended.

```
sbatch --dependency=afterany:$jid cleanup.sh
```

ADVANCED TIPS · MANY RUNS

Job arrays

Same job, many seeds, settings or data chunks: one script, 16 runs.

```
#SBATCH --array=1-16%8
#SBATCH --output=slurm_%A_%a.out

python train.py --seed $SLURM_ARRAY_TASK_ID
```

%8 runs at most 8 tasks at once.

%A is the job, %a the task number.

Chunked preprocessing or downloads: same cost, just faster.

KEEP THESE HANDY

Links

PARTITIONS	Snellius partitions and SBU weights
JOB SCRIPT	Writing a job script
COPY DATA	Transfer files between systems
DISK USAGE	Checking disk usage
TUTORIAL	UvA DL: working on the cluster
LOGIN	Connecting to the system
HELP	SURF Service Desk